

SESSION 7 MINI-ASSESSMENT REPORT

Design and Implementation of a Multi-Step Prompt Workflow for Resume Screening

Submitted by

Name: K. Niharika

Registration No:99230040633

Programme: B.Tech CSE (AI & ML)

Module: Prompt Engineering

Session: 7 – Multi-Step Prompt Workflow

Date of Submission: 05-09-26

Table of Contents

1. Introduction	
2. Objective	
3. Workflow Design	
4. Methodology	
4.1 Step 1: Candidate Profile Extraction Using Few-Shot Prompting.....	
4.2 Step 2: Job-Fit Assessment Using Chain-of-Thought Reasoning.....	
4.3 Step 3: Structured Output Using JSON	
5. Results and Discussion	
6. Conclusion.....	

1. Introduction

Recruiters and hiring managers routinely receive dozens to hundreds of resumes for a single opening, each formatted differently and phrased in the applicant's own words. Manually screening every resume against a job description does not scale, and manual screening is also vulnerable to fatigue and inconsistency — two resumes with equivalent qualifications can be judged differently depending on when in the pile they were read. This report documents the design and implementation of a multi-step prompt workflow that automates the first pass of resume screening using a large language model (LLM), breaking the task into three chained stages rather than relying on a single, overloaded prompt.

The workflow was built and tested using representative sample resumes and job descriptions spanning software engineering, data analysis, and marketing roles, and demonstrates how prompt chaining, few-shot examples, chain-of-thought reasoning, and structured JSON output can be combined into a single dependable pipeline for candidate shortlisting.

2. Objective

The objective of this mini-assessment is to design, implement, and evaluate a multi-step prompt workflow that takes a raw resume and a job description as input and produces a structured screening decision as output. Specifically, the workflow aims to:

- Extract key candidate attributes from an unstructured resume (skills, years of experience, education, and past job titles).
- Reason step-by-step about how well the candidate matches the job description rather than assigning a fit score directly.
- Return the final screening decision in a strict, machine-readable JSON format suitable for direct ingestion into an applicant tracking system (ATS).
- Compare the reliability of this chained approach against a single monolithic prompt attempting the same task in one step.

3. Workflow Design

A single prompt that asks a model to "read this resume and tell me if the candidate is a good fit" tends to produce shallow, inconsistent answers because it forces extraction, reasoning, and formatting to happen simultaneously inside one pass, and it makes the basis for a rejection difficult to justify later. The design adopted here instead treats resume screening as a pipeline of three specialised stages, where the output of each step becomes the input to the next:

1. Candidate Profile Extraction (few-shot prompting) — pulls structured facts out of the raw resume text.
2. Job-Fit Assessment (chain-of-thought reasoning) — reasons over the extracted profile against the job description to judge suitability.
3. Structured Output Generation (JSON schema prompting) — converts the reasoning into a strict JSON object for downstream systems.

This separation of concerns mirrors good software engineering practice: each stage has one responsibility, can be tested independently, and produces an intermediate artefact that a human recruiter can inspect if the final decision looks wrong — which matters here given how consequential and personal a rejection decision is.

4. Methodology

The methodology section walks through each of the three pipeline stages, including the exact prompt structure used, a worked example, and the reasoning behind key design choices.

4.1 Step 1: Candidate Profile Extraction Using Few-Shot Prompting

The first stage converts an unstructured resume into a small set of named fields: `candidate_name`, `total_years_experience`, `education_level`, `key_skills`, and `most_recent_title`. A few-shot prompt was used rather than a zero-shot instruction because early testing showed that zero-shot extraction was inconsistent about how it counted overlapping job dates toward total experience, and about which bullet points counted as a "skill" versus a generic responsibility. Providing two worked examples inside the prompt anchored the model's output format and counting logic.

SYSTEM: You are a resume-parsing assistant. Extract only the fields listed below. If a field is not mentioned, output null.

Example 1

Resume excerpt: "Anita Rao. B.Tech Computer Science, 2019–2023. Software Engineer at Zeta Labs (2023–present): built REST APIs in Python and Django, worked with PostgreSQL and Docker."
Output: {`candidate_name`: Anita Rao, `total_years_experience`: 1, `education_level`: Bachelor's, `key_skills`: [Python, Django, PostgreSQL, Docker], `most_recent_title`: Software Engineer}

Example 2

Resume excerpt: "Karthik Iyer. MBA Marketing, 2015. Marketing Manager at BrightAds (2016–2022), then Senior Marketing Manager at Loom Media (2022–present). Skilled in SEO and campaign analytics."
Output: {`candidate_name`: Karthik Iyer, `total_years_experience`: 8, `education_level`: Master's, `key_skills`: [SEO, campaign analytics], `most_recent_title`: Senior Marketing Manager}

Now extract the same fields from this resume:
"{{raw_resume_text}}"

Two examples were deliberately chosen to cover contrasting cases — an early-career technical candidate and a senior non-technical candidate — so the model would learn to compute `total_years_experience` by summing distinct role durations rather than copying a single date range. Testing with only one example (the junior candidate) caused the model to under-count experience for resumes listing multiple sequential roles in early trial runs.

4.2 Step 2: Job-Fit Assessment Using Chain-of-Thought Reasoning

The second stage takes the structured profile from Step 1 and a target job description, then reasons about fit before committing to a recommendation. Asking the model to jump straight to a fit label ("Shortlist / Reject") produced plausible-looking but poorly justified answers, so the prompt instead requires an explicit reasoning trace covering three questions: whether required skills are present, whether experience level meets the role's minimum, and whether there are any clear gaps worth flagging to a recruiter.

SYSTEM: You are assessing candidate fit for a role. Think step by step before giving a final recommendation.

Candidate profile: {profile_from_step_1}
Job description: {job_description_text}

Answer in order:

1. Which required skills from the job description does the candidate's profile clearly satisfy?
2. Does the candidate's experience meet the role's minimum experience requirement?
3. What gaps, if any, should a recruiter be aware of?
4. Based on 1–3, recommend: Strong Match, Possible Match, or Not a Match.

Provide the reasoning for each question, then the final recommendation on its own line prefixed with 'RECOMMENDATION:'.

Requiring the intermediate reasoning steps served two purposes beyond accuracy: it gave a human recruiter a transparent justification for every recommendation made, and it made it possible to catch cases where the model's own reasoning contradicted its final label — a strong signal to flag the candidate for manual review rather than auto-rejecting on the model's output alone.

4.3 Step 3: Structured Output Using JSON

The final stage reformats the fit recommendation and supporting profile into a single JSON object matching the schema expected by the applicant tracking system's API. The prompt fixes the exact key names, types, and an enumerated list of allowed values for `education_level` and `recommendation`, and instructs the model to return only the JSON object with no surrounding commentary.

SYSTEM: Convert the screening result into JSON only — no other text.

Schema:

```
{
  "candidate_name": string | null,
  "total_years_experience": number | null,
  "education_level": "High School" | "Bachelor's" | "Master's" | "PhD" | "Other",
  "recommendation": "Strong Match" | "Possible Match" | "Not a Match",
  "reasoning_summary": string
}
```

Input: {profile_and_reasoning_from_steps_1_and_2}

Constraining `education_level` and `recommendation` to an enumerated set — rather than letting the model invent free-text labels — was essential for reliable downstream parsing; without it,

near-duplicate labels such as "Master's Degree" and "MBA/Master's" appeared across otherwise identical candidate profiles.

5. Results and Discussion

The chained workflow was compared against a single-prompt baseline that asked for candidate extraction, fit assessment, and JSON output all in one instruction. Across the sample resumes tested against several job descriptions, the multi-step pipeline produced noticeably more consistent and defensible results.

Criterion	Single-Prompt Baseline	Multi-Step Workflow
Experience-counting accuracy	Miscounted overlapping/sequential roles	Consistent summation logic across resumes
Recommendation justification	Label only, no reasoning	Explicit, auditable reasoning trace
Output parseability	Occasional extra commentary broke JSON parsing	Clean JSON on every run tested
Handling of borderline candidates	Defaulted to outright reject	Reasoning surfaced gaps for recruiter review

The main trade-off observed was latency and cost: three chained model calls take longer and consume more tokens in total than one call. For a resume-screening system, this trade-off is acceptable because screening happens once per application and the accuracy and auditability gains reduce costly downstream errors, such as a qualified candidate being auto-rejected without a recoverable justification.

A second observation was that few-shot examples in Step 1 had a larger effect on overall pipeline quality than any change made to Steps 2 or 3, since an incorrectly extracted experience total or skill list propagated forward and skewed the final recommendation. This suggests that in a chained workflow, investing effort in the earliest, most upstream step yields the highest return. A further consideration specific to this domain is fairness: because the reasoning trace in Step 2 is fully visible, it is possible to audit whether the model is basing its recommendation on job-relevant criteria rather than on incidental details such as the candidate's name, university, or the formatting of their resume.

6. Conclusion

Breaking resume screening into three focused prompts — candidate profile extraction, chain-of-thought job-fit assessment, and structured JSON generation — produced more consistent, explainable, and machine-readable results than a single all-in-one prompt. The approach also introduced natural checkpoints where a human recruiter can validate or override the model at each stage, which matters for a task where a wrong rejection can have real consequences for a candidate and real legal and reputational exposure for the employer.

Future extensions to this workflow could include a fourth step that ranks shortlisted candidates relative to one another for a given role, feedback loops that let recruiter corrections at any stage

automatically improve the few-shot examples used in Step 1, an explicit bias-check pass over the Step 2 reasoning trace, and evaluation against a larger, more diverse resume dataset to quantify accuracy more rigorously than the qualitative comparison presented here.